

3. 보수의 표현

- 1) 컴퓨터에서의 데이터 표현
- 2) 보수 표현을 사용하는 이유
- 3) 보수의 10진수 변환

3. 보수의 표현

- [예] 3에 대한 7의 보수: $3+n=7 \quad \therefore n=4$
- 1의 보수(1' complement) 표현
 - 어떤 수 n 과의 합이 1이 되는 수
 - 예: 2진수 00110101의 1의 보수 = 11001010
- 2의 보수(2' complement) 표현
 - 어떤 수 n 과의 합이 2가 되는 수
 - 예: 2진수 00110101의 2의 보수 = 11001011

3-1) 컴퓨터에서의 데이터 표현-1

• 부호화-절대치 표현

- 부호와 데이터의 절대값을 그대로 표현

최상위 비트	...	...	...	...	...	...	...	...	...	...	...	...	...	...	최하위 비트
부호비트	절대치 비트(데이터 비트)														

- [예제 4-28] 10진수 +53과 -53에 대하여 8bit 부호화-절대치 표현으로 나타내시오.

$$+53_{10} = \textcolor{red}{0}0110101_2$$

$$-53_{10} = \textcolor{red}{1}0110101_2$$

0	0	1	1	0	1	0	1
부 호 (양수)	$ 53_{10} = 110101_2$						

1	0	1	1	0	1	0	1
부 호 (음수)	$ 53_{10} = 110101_2$						

3-1) 컴퓨터에서의 데이터 표현-2

- ①1의 보수 / ②2의 보수 표현에 대한 특징
 - 〈공통점〉
 - 음수 표현에만 적용
 - 음수에 대한 부호화-절대치 표현에서 부호 비트는 그대로 사용
 - 〈차이점〉 음수에 대한 부호화-절대치 표현에서 절대치 비트(데이터 비트)에 대해 ...
 - ① 1의 보수(0은 1로, 1은 0으로)로 바꿈
 - ② 2의 보수(1의 보수에 1을 더함)로 바꿈

3-1) 컴퓨터에서의 데이터 표현-3

▪ [예제] +53과 -53의 8bit 1의 보수/2의 보수 표현

(1) +53은 부호화-절대치 표현과 동일 (:: 양수)

$\therefore 00110101$

(2) -53은 부호 비트를 제외한 7bit 데이터 비트는

0110101_2 이므로

부호화-절대치 표현	0 0110101
① 1의 보수 표현	11001010
	<u>+</u> 1
② 2의 보수 표현	11001011

3-1) 컴퓨터에서의 데이터 표현-4

• 데이터의 범위(1)

▪ n 비트를 사용하는 컴퓨터에서

-1의 보수: $-(2^{n-1}-1) \sim (2^{n-1}-1)$

(-0) 1111	(-1) 1110	(-2) 1101	(-3) 1100	(-4) 1011	(-5) 1010	(-6) 1001	(-7) 1000
(0) 0000	(1) 0001	(2) 0010	(3) 0011	(4) 0100	(5) 0101	(6) 0110	(7) 0111

-2의 보수: $-2^{n-1} \sim (2^{n-1}-1)$

(-1) 1111	(-2) 1110	(-3) 1101	(-4) 1100	(-5) 1011	(-6) 1010	(-7) 1001	(-8) 1000
(0) 0000	(1) 0001	(2) 0010	(3) 0011	(4) 0100	(5) 0101	(6) 0110	(7) 0111

3-1) 컴퓨터에서의 데이터 표현-5

• 데이터의 범위(2)

- 연산 결과가 데이터의 범위를 벗어나는 경우를 초과(overflow)라고 하며 다음 2가지 경우에 발생

- ① 두 개의 양수를 더했을 때 음수가 나오는 경우
- ② 두 개의 음수를 더했을 때 양수가 나오는 경우

- [예] 4bit 부호화-절대치 표현에서

- ① $4_{10} + 5_{10} = 0100_2 + 0101_2 = 1001_2 = -1_{10} \neq 9_{10}$
- ② $(-3) + (-4) = 1011 + 1100 = 10111 = 7 \neq -7$

3-2) 보수 표현을 사용하는 이유

- 부호화-절대치 표현의 한계
 - 연산의 결과가 부정확: 초과 발생 시
 - 예: 4bit에서 연산: $4+5$, $(-3)+(-4)$
 - 0에 대한 2가지 표현: $0000(+0)$, $1000(-0)$
- 1의 보수 표현의 한계
 - 연산의 정확성('4. 보수의 연산'에서 설명)
 - 0에 대한 2가지 표현: $0000(+0)$, $1111(-0)$
- 2의 보수 표현에서 위의 2가지 문제 모두 해결
 - 연산의 정확성, 0에 대한 유일한 표현

3-3) 보수의 10진수 변환

- 1의 보수(2의 보수)를 10진수로 변환하는 방법

- $-(2^{n-1}-1)(-2^{n-1}) + \text{절대치 비트의 10진수 표현}$
- 1의 보수(2의 보수)를 다시 1의 보수(2의 보수)로 변환한 후 이에 대해 10진수 표현

- [예제] 4bit 컴퓨터에서 다음을 10진수로 변환

-1의 보수 1010: ① $-(2^{4-1}-1) + (1 \times 2^1) = -5$

② $1101_2 = -(1 \times 2^2 + 1 \times 2^0) = -5$

-2의 보수 1011: ① $-(2^{4-1}) + (1 \times 2^1 + 1 \times 2^0) = -5$

② $1101_2 = -(1 \times 2^2 + 1 \times 2^0) = -5$

4. 보수의 연산-1

- [예제 ①] 4bit 컴퓨터에서 $2_{10}-4_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오

$$|2_{10}|=0010, |4_{10}|=0100$$

(1')	0010
+	1011
	1101

(2')	0010
+	1100
	1110

-1의 보수 이용: $1101_2 \rightarrow 1010 = -2_{10}$

-2의 보수 이용: $1110_2 \rightarrow 1010 = -2_{10}$

4. 보수의 연산-2

- [예제 ②] 4bit 컴퓨터에서 $4_{10}-2_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오

$$|4_{10}|=0100, |2_{10}|=0010$$

(1')	0100
+	1101
	10001
+	1
	0010

(2')	0100
+	1110
	10010
(무시)	

-1의 보수 이용: $0010_2=2_{10}$

-2의 보수 이용: $0010_2=2_{10}$

4. 보수의 연산-3

- [예제 ③] 4bit 컴퓨터에서 $-2_{10}-4_{10}$ 에 대해 1의 보수와 2의 보수를 이용하여 연산하시오

$$|2_{10}|=0010, |4_{10}|=0100$$

(1')	1101
+	1011
	11000
+	1
	1001

(2')	1110
+	1100
	11010
	(무시)

-1의 보수 이용: $1001_2 \rightarrow 1110 = -6_{10}$

-2의 보수 이용: $1010_2 \rightarrow 1110 = -6_{10}$

2-2) 수 체계의 변환-1

10진수 14.75를 2진수로 변환

2	14		최하위
2	7	...	0
2	3	...	1
2	1	...	1
2	0	...	1
			최상위

최상위자리
↓
 $\therefore 14 = 1110$
↑
최하위자리

	0.75		0.5
×	2	×	2
	1.50		1.0

$\therefore 0.75 = 0.11$

4. 보수의 연산

- 4bit 컴퓨터에서 $4_{10} - 2_{10}$ 에 대해
1의 보수와 2의 보수를 이용한 연산
($|4_{10}| = 0100$, $|2_{10}| = 0010$)

(1')	0100
+	1101
	10001
+	1
	0010

(2')	0100
+	1110
	10010
	(무시)

3-3) 보수의 10진수 변환

• n 비트 1의 보수를 10진수로 변환하는 방법

① $-(2^{n-1}-1) + \text{절대치 비트의 10진수 표현}$

(-0)	(-1)	(-2)	(-3)	(-4)	(-5)	(-6)	(-7)
1111	1110	1101	1100	1011	1010	1001	1000
(0)	(1)	(2)	(3)	(4)	(5)	(6)	(7)
0000	0001	0010	0011	0100	0101	0110	0111

② 1의 보수로 다시 변환 후 10진수 표현

▪ 1의 보수 1010(4bit) → 10진수 표현

① $-(2^{4-1}-1) + (1 \times 2^1) = -5$

② $1101_2 = -(1 \times 2^2 + 1 \times 2^0) = -5$